

Lock(interface)

04 April 2026 10:16

The problems with traditional synchronized Keyword are as follows:

1. We are not having any flexibility to try for the lock without waiting.
2. There is no way to specify maximum waiting time for a thread to get locked. So that thread will wait until getting the lock which may create performance problems which may cause deadlock.
3. If a thread releases the lock then which waiting thread will get that lock, We are not having any control on this.
4. There is no API to list out all waiting thread for a lock.
5. Synchronized keyword compulsory we have to use either at method level or within the method and it is not possible to use across multiple methods.

To overcome these problems Sun Microsystem(now Oracle Inc.) introduces `java.util.concurrent.locks` package in version 1.5. It also provides several enhancement to the programmer to provide more control on concurrency.

Lock(I)

Lock(I): Lock object is similar to implicit lock acquired by a thread to execute synchronized method or synchronized block. Lock implementation. Provide more extensive operations than traditional implicit locks.

Important methods of Lock(I):

1. **void lock():** So we can use this method to acquire Lock. If a lock is already available then immediately current thread will get that lock. If the lock is not already available then it will wait until getting the lock. It is exactly has same behavior of traditional synchronized keyword.

2. **boolean tryLock():**

```
if(l.tryLock()){  
    //perform the safe operations  
}  
else{  
    //perform alternative operations  
}
```

To acquire the lock without waiting. If the lock is available, then the thread acquires the lock and returns true. If the lock is not available then this method returns false and can continue its execution without waiting. In this case thread never be entered into waiting state. Now the third method.

3. **boolean tryLock(long time, TimeUnit unit):** If lock is available then the thread will get the lock and continue its execution. If the lock is not available, then the thread will wait until a specified amount of time. Still, if the lock is not available, then the thread can continue its execution.

TimeUnit: is an enum(meaning a group of constant)
`java.util.concurrent.TimeUnit`

```
enum TimeUnit{  
    NANOSECONDS,  
    MICROSECONDS,  
    MILLISECONDS  
    SECONDS,  
    MINUTES,  
    HOURS,
```

```

    DAYS
}

if (l.tryLock(1000, TimeUnit.MILLISECONDS)){
    ...
}

```

4. **void lockInterruptibly():** Acquires lock if it is available and returns immediately. If the lock is not available then it will wait. While waiting, if the thread is interrupted then the thread won't get the lock.

5. **void unlock():** It releases the lock.

```

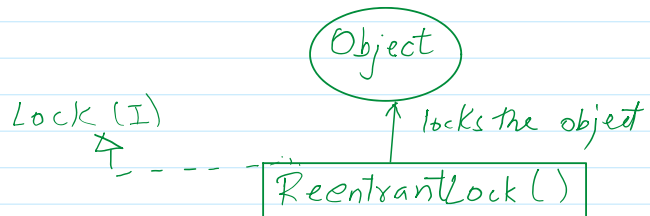
l.lock();
...
...
l.unlock();

```

It throws runtime exception saying: `IllegalMonitorStateException`.

To call this method compulsorily, current thread should be the owner of the lock. Otherwise we will get runtime exception saying `IllegalMonitorStateException`.

6. ReentrantLock:



It is a implementation of Lock interface. And it is a direct child class of Object.

```

public class ReentrantLock
extends Object
implements Lock, Serializable

```

A reentrant mutual exclusion [Lock](#) with the same basic behavior and semantics as the implicit monitor lock accessed using synchronized methods and statements, but with extended capabilities.

A `ReentrantLock` is *owned* by the thread last successfully locking, but not yet unlocking it. A thread invoking lock will return, successfully acquiring the lock, when the lock is not owned by another thread. The method will return immediately if the current thread already owns the lock. This can be checked using methods [isHeldByCurrentThread\(\)](#), and [getHoldCount\(\)](#).

The constructor for this class accepts an optional *fairness* parameter. When set true, under contention, locks favor granting access to the longest-waiting thread. Otherwise this lock does not guarantee any particular access order. Programs using fair locks accessed by many threads may display lower overall throughput (i.e., are slower; often much slower) than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of starvation. Note however, that fairness of locks does not guarantee fairness of thread scheduling. Thus, one of many threads using a fair lock may obtain it multiple times in succession while other active threads are not progressing and not currently holding the lock. Also note that the untimed [tryLock\(\)](#) method does not honor the fairness setting. It will succeed if the lock is available even if other threads are waiting.

It is recommended practice to *always* immediately follow a call to lock with a try block,

most typically in a before/after construction such as:

```
class X {
    private final ReentrantLock lock = new ReentrantLock();
    // ...
    public void m() {
        lock.lock(); // block until condition holds
        try {
            // ... method body
        } finally {
            lock.unlock()
        }
    }
}
```

In addition to implementing the [Lock](#) interface, this class defines a number of public and protected methods for inspecting the state of the lock. Some of these methods are only useful for instrumentation and monitoring.

Serialization of this class behaves in the same way as built-in locks: a deserialized lock is in the unlocked state, regardless of its state when serialized.

This lock supports a maximum of 2147483647 recursive locks by the same thread. Attempts to exceed this limit result in [Error](#) throws from locking methods.

Since:

1.5

Reentrant means a thread can acquire same lock multiple times without any issue.

Internally Reentrant lock increment threads personal count whenever we call lock () method. Decrements count value whenever thread calls unlock () method. And lock will be released whenever count reaches zero.

```
ReentrantLock l = new ReentrantLock();
```

```
l.lock(); //holdCount = 1
```

```
l.lock(); //holdCount = 2
```

```
l.lock(); //holdCount = 3
```

```
l.unlock(); //holdCount = 2
```

```
l.unlock(); //holdCount = 1
```

```
l.unlock(); //holdCount = 0
```

Constructors

Constructor	Description
<code>ReentrantLock()</code>	Creates an instance of <code>ReentrantLock</code> .
<code>ReentrantLock(boolean fair)</code>	Creates an instance of <code>ReentrantLock</code> with the given fairness policy.

↑

Creates `ReentrantLock` with the given fairness policy. If fairness is true. then longest waiting thread can acquire the lock, if it is available, i.e., it follows first come first serve policy (FCFS). If fairness is false. then

Which waiting thread will get the chance, we can't expect.

Note: The default value of fairness is false..

Which of the following declarations are equal.

- ~~(i)~~ ReentrantLock l = new ReentrantLock();
- (ii) ReentrantLock l = new ReentrantLock(true);
- ~~(iii)~~ ReentrantLock l = new ReentrantLock(false);
- (iv) All the above.

Important methods of ReentrantLock:

```
void lock()
boolean tryLock()
boolean tryLock(long l, TimeUnit t)
void lockInterruptibly()
void unlock()
```

int getHoldCount(): The returns number of holds on this lock by current thread.

boolean isHeldByCurrentThread(): Returns true if and only if lock is hold by current thread.

int getQueueLength(): Returns number of threads waiting for the lock.

Collection getQueueThreads(): It returns a collection of threads which are waiting to get lock.

boolean hasQueuedThreads(): Returns true if any thread waiting to get the lock.

boolean isLocked(): Returns true if the lock is acquired by some thread.

boolean isFair(): Returns true if the fairness policy is set with true value.

Thread getOwner(): Returns the thread which acquires the lock. So let me show you one small program. Today.

```
package MyThreads.MultithreadingEnhancement;
import java.util.concurrent.locks.ReentrantLock;
public class ReentrantLockDemo {
    public static void main(String[] args) {
        ReentrantLock l = new ReentrantLock();
        l.lock();
        l.lock();
        System.out.println(l.isLocked()); //true
        System.out.println(l.isHeldByCurrentThread()); //true
        System.out.println(l.getQueueLength()); //0
        l.unlock();
        l.unlock();
        System.out.println(l.isLocked()); //false
    }
}
```

```

        System.out.println(l.isFair()); //false
    }
}

```

Output:

```

true
true
0
false
false

```

```

package MyThreads.MultithreadingEnhancement;
public class Display {
    public synchronized void wish(String name) throws InterruptedException{
        for (int i = 0; i < 10; i++) {
            System.out.print("Good morning: ");
            Thread.sleep(2000);
            System.out.println(name);
        }
    }
}

```

```

package MyThreads.MultithreadingEnhancement;
public class DisplayThread extends Thread{
    private Display d;
    private String name;
    public DisplayThread(Display d, String name){
        this.d = d;
        this.name = name;
    }
    @Override
    public void run() {
        try {
            d.wish(name);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

```

package MyThreads.MultithreadingEnhancement;
public class SynchronizedDemo {
    public static void main(String[] args) {
        Display d = new Display();

        DisplayThread t1 = new DisplayThread(d, "Dhoni");
        DisplayThread t2 = new DisplayThread(d, "Surya");
        t1.start();
        t2.start();
    }
}

```

The modified version of Display class, rest is same:

```
package MyThreads.MultithreadingEnhancement;
import java.util.concurrent.locks.ReentrantLock;
public class Display {
    private ReentrantLock rl = new ReentrantLock();
    public void wish(String name) throws InterruptedException{
        rl.lock();          //1
        for (int i = 0; i < 10; i++) {
            System.out.print("Good morning: ");
            Thread.sleep(2000);
            System.out.println(name);
        }
        rl.unlock(); //2
    }
}
```

If we comment line one and two then the threads will be executed simultaneously and we will get irregular output. In the above code we have removed synchronized keyword from our code.

```
package MyThreads.MultithreadingEnhancement;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;
public class MyReentrantLockThread extends Thread{
    private static ReentrantLock rl = new ReentrantLock();
    public MyReentrantLockThread(String name){
        super(name);
    }
    @Override
    public void run() {
        do{
            try {
                if(rl.tryLock(5000, TimeUnit.MILLISECONDS)){
                    System.out.println(Thread.currentThread().getName() +
                        " ... got the lock and performing safe operations");
                    try {
                        Thread.sleep(5000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    rl.unlock();
                    break;
                }
            } else{
                System.out.println(Thread.currentThread().getName() +
                    " ... unable to get the lock and performing alternative operations");
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }while(true);
}
}
```

```
package MyThreads.MultithreadingEnhancement;
public class MyReentrantLockDemo {
    public static void main(String[] args) {
        MyReentrantLockThread t1 = new MyReentrantLockThread("First thread");
        MyReentrantLockThread t2 = new MyReentrantLockThread("Second thread");
        t1.start();
        t2.start();
    }
}
```